

UNIVERSIDAD TECNOLÓGICA DE TORREÓN

Plataformas de Servicios en la Nube

Docente:

M.C. Rubén Hernández Barraza

Instalación de Docker Desktop y Despliegue de API Local

Jesus Andrés Roldan Hernández

Grado y Sección: 4to MIMI

Torreón, Coahuila a 17 de septiembre del 2026

ÍNDICE

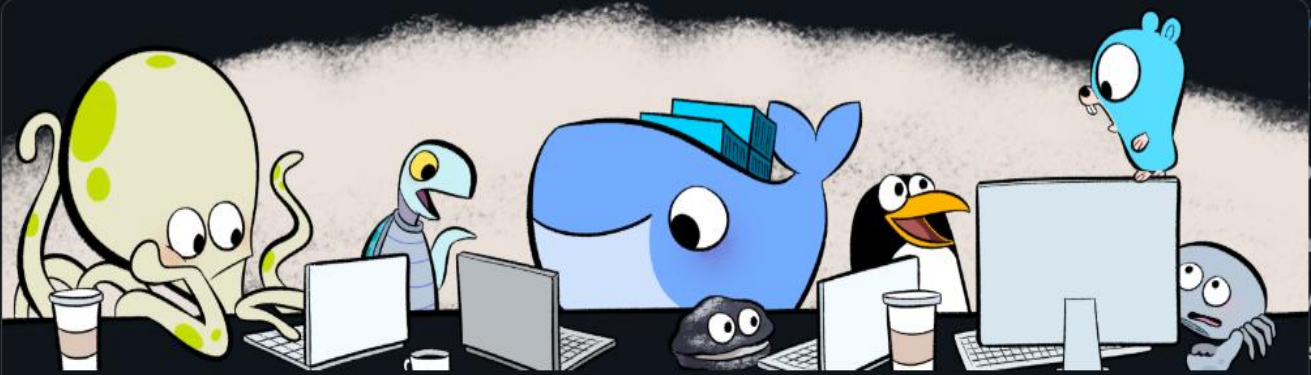
CONTENIDO	3
Objetivo de la practica	3
Desarrollo	4
Conclusión	17

CONTENIDO

Objetivo de la practica

La instalación de un entorno de contenedores “Docker Desktop” y empaquetar Nuestra API creada en la practica 1, mediante Dockerfile para su despliegue y ejecución aislada.

Inmutabilidad de los contenedores: Una vez construida la imagen a partir del Dockerfile, esta debe considerarse inmutable. Cualquier actualización en la API o cambio en las dependencias requiere construir una nueva versión de la imagen, garantizando que el entorno de ejecución sea idéntico y consistente en los entornos de desarrollo, pruebas y producción.



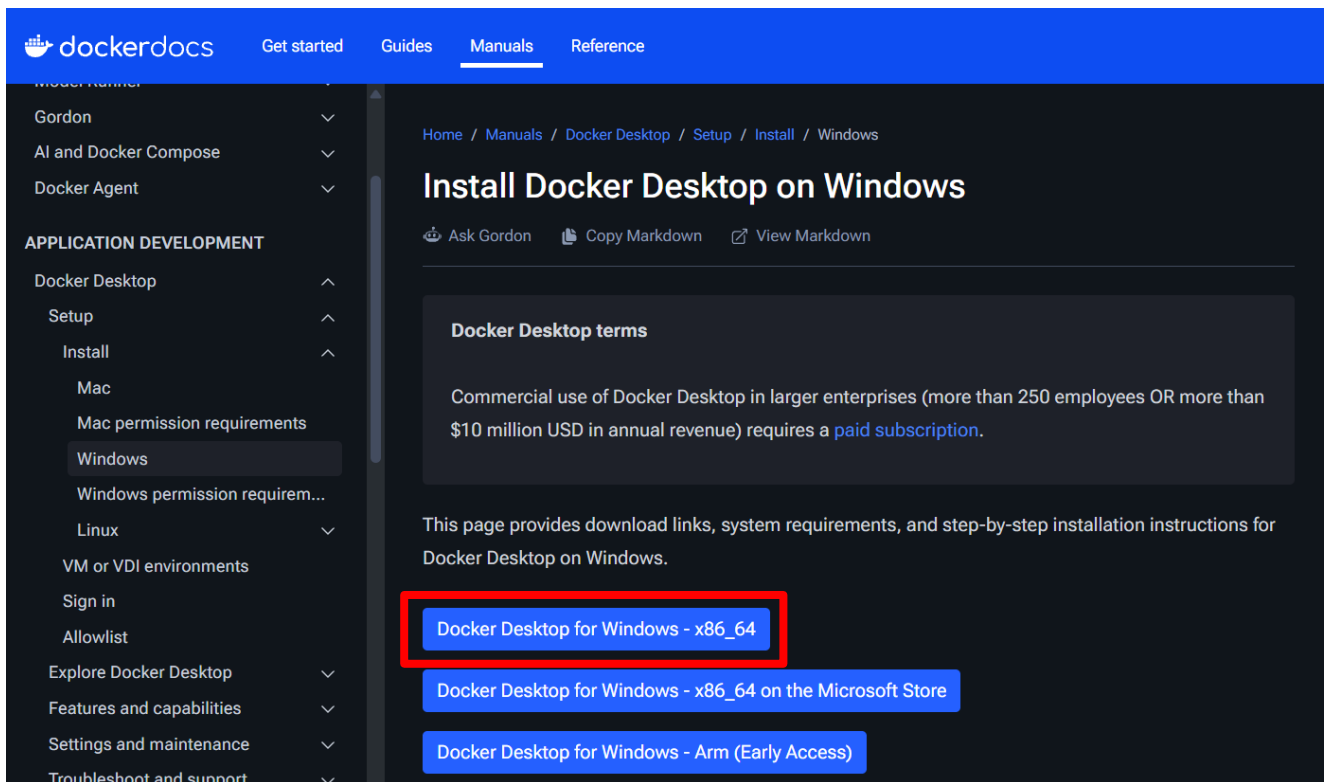
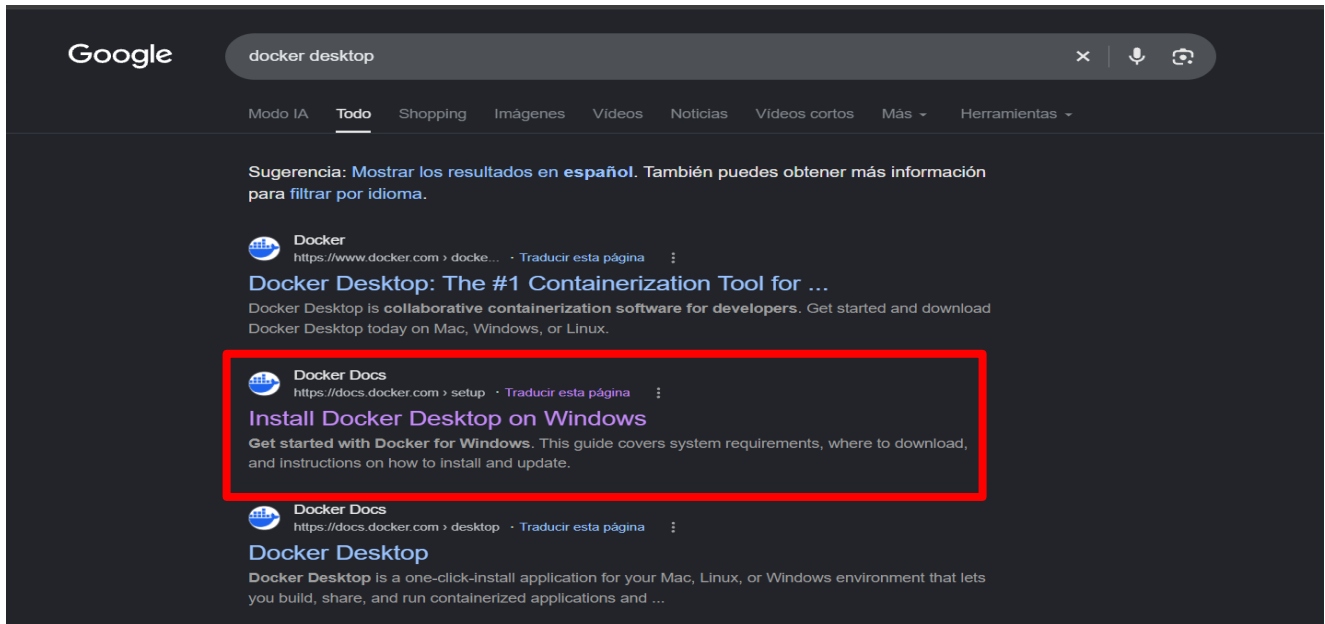
Docker Subscription Service Agreement

By selecting accept, you agree to the [Subscription Service Agreement](#), the [Docker Data Processing Agreement](#), the [Data Privacy Policy](#) and the [Docker AI Supplemental Terms](#).

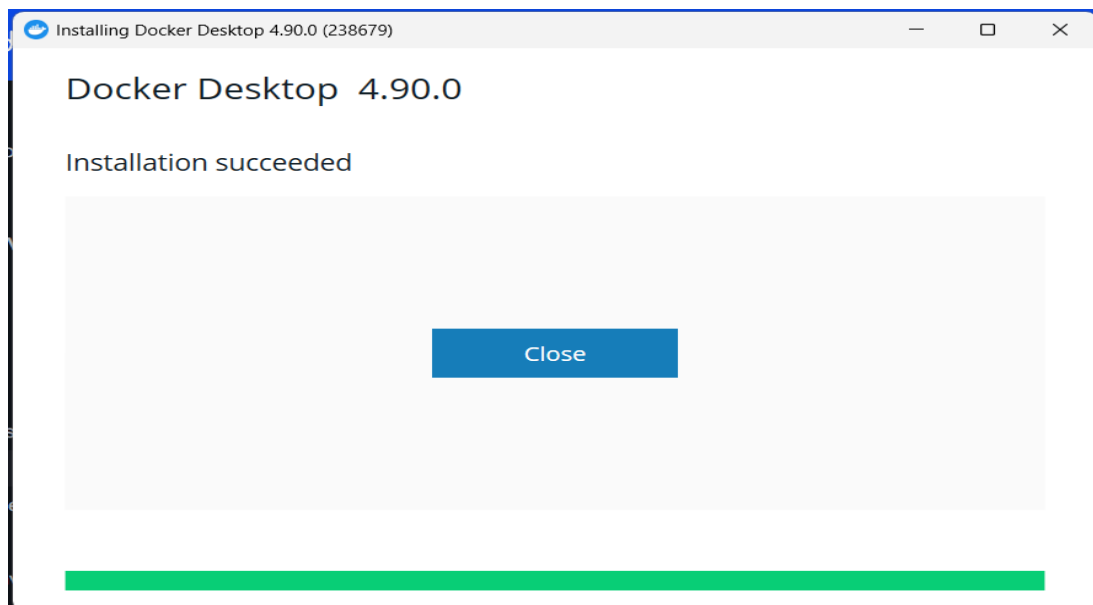
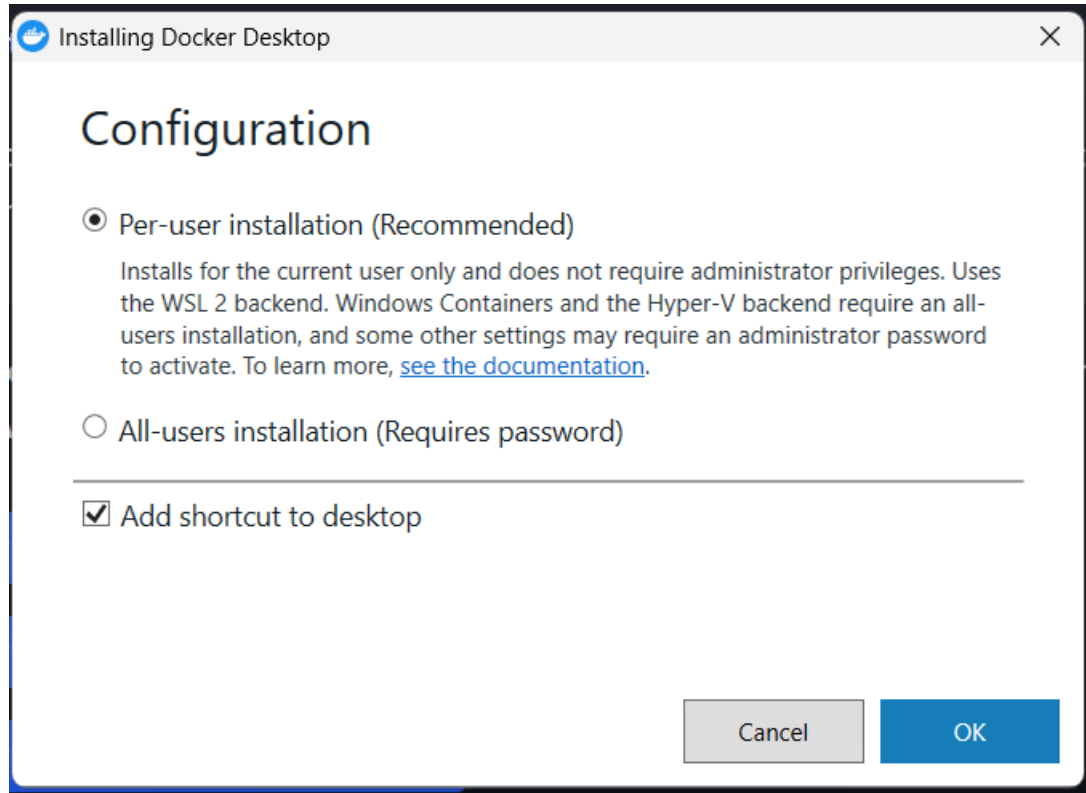
Commercial use of Docker Desktop at a company of more than 250 employees OR more than \$10 million in annual revenue requires a paid subscription (Pro, Team, or Business). [See subscription details](#)

Desarrollo

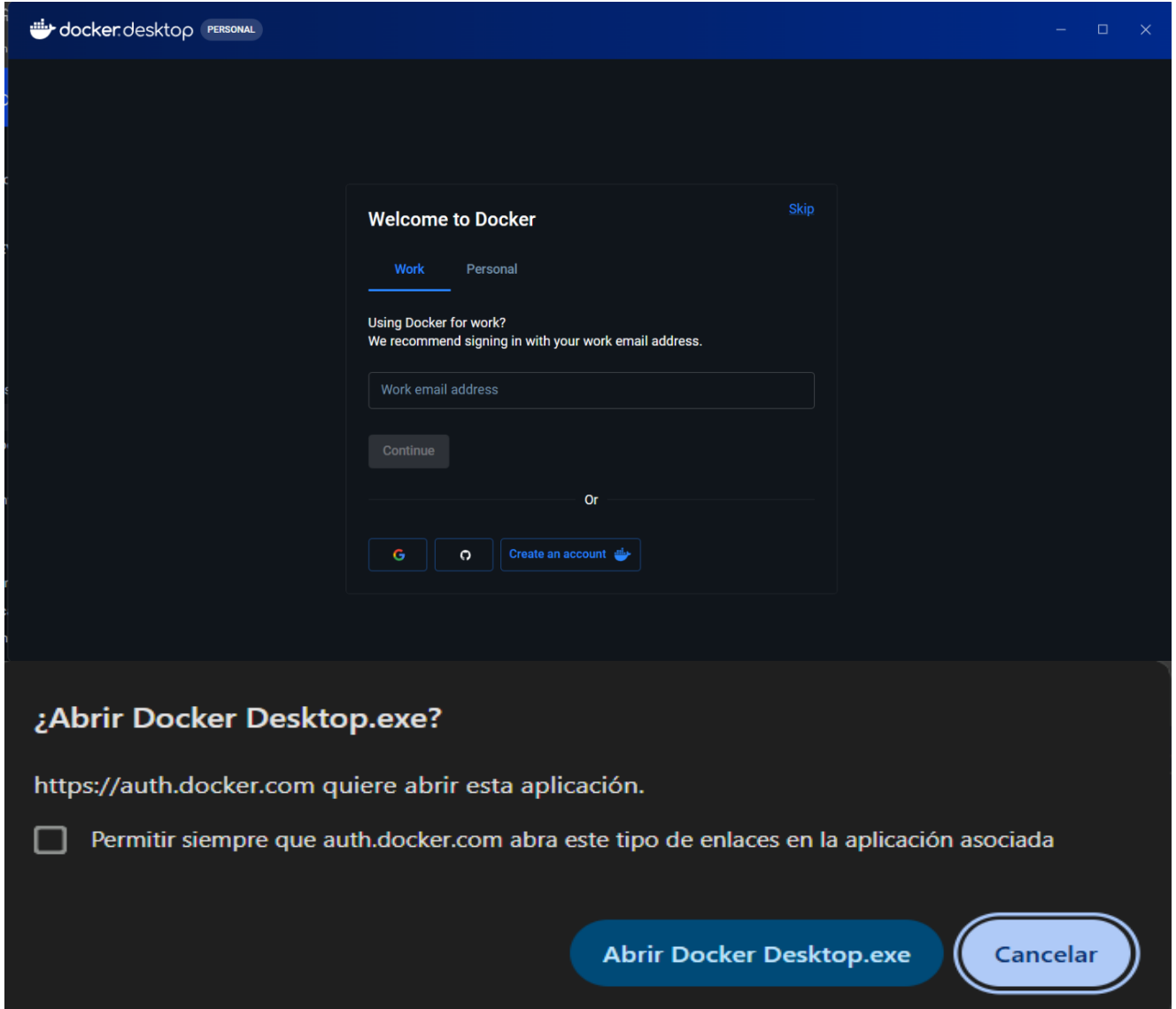
Iniciamos con la búsqueda de Docker Desktop, en el navegador de nuestra preferencia y entramos en el enlace señalado “Docker Docs”



Descargamos el ejecutable y continuamos con la instalación seleccionando la opción recomendada (si estás en Windows, asegúrate de tener habilitada la integración con WSL 2) y esperamos a finalizar y reiniciamos el equipo.



Continuamos abriendo el programa Docker desde la barra de búsqueda o el acceso directo que se creó durante la instalación, entramos con nuestra cuenta de Google y le damos los permisos requeridos.



The image shows two overlapping windows. The background window is the Docker Desktop application, which has a dark blue header with the 'docker.desktop' logo and a 'PERSONAL' tab. The main content area is dark gray and displays a 'Welcome to Docker' dialog. This dialog has two tabs: 'Work' (selected) and 'Personal'. Under the 'Work' tab, it asks 'Using Docker for work?' and recommends signing in with a work email address. There is a text input field for the 'Work email address' and a 'Continue' button. Below this, it says 'Or' and provides three options: a Google icon, a Docker icon, and a 'Create an account' button with a Docker icon. A 'Skip' link is in the top right corner of the dialog.

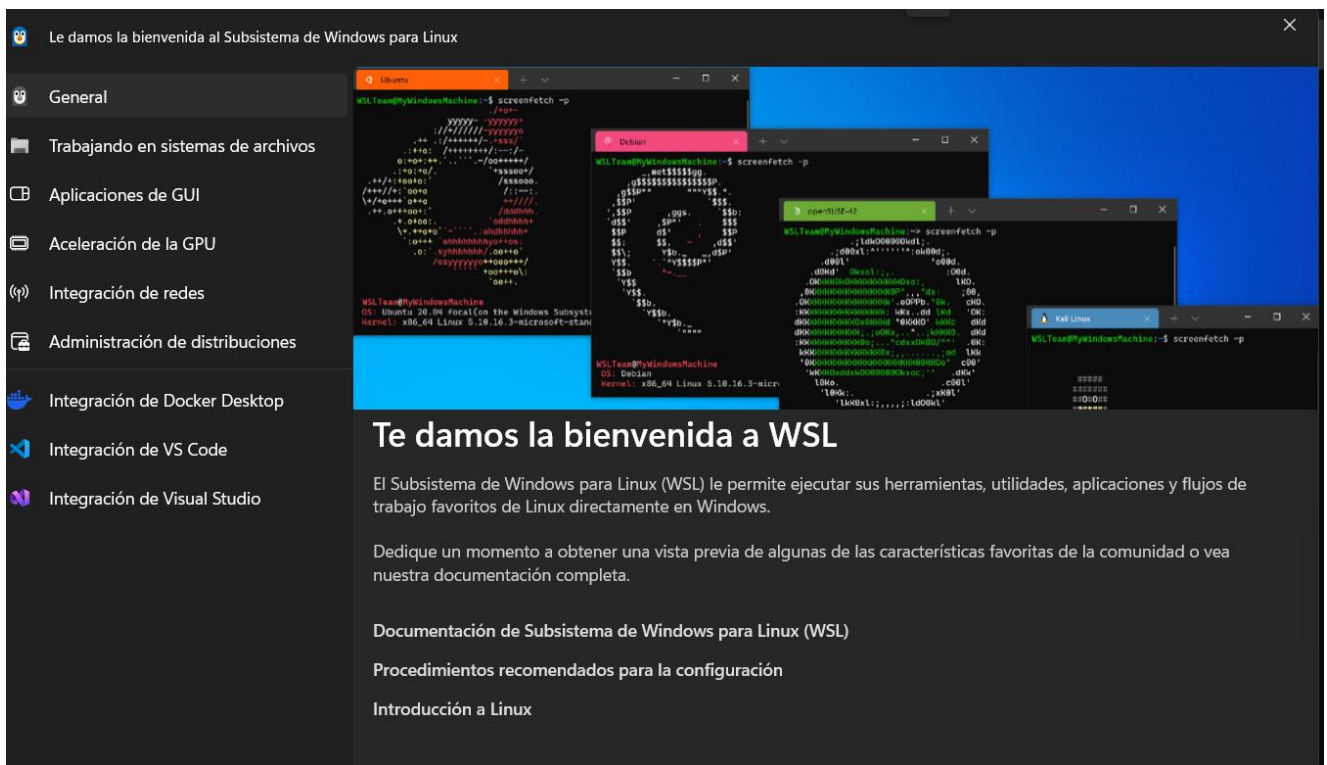
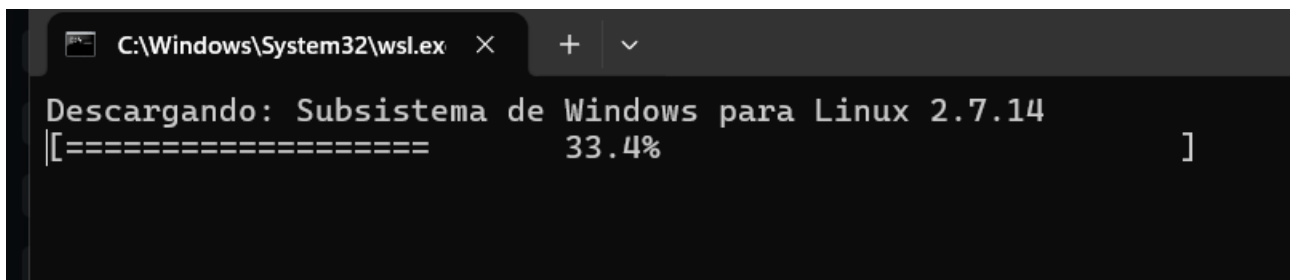
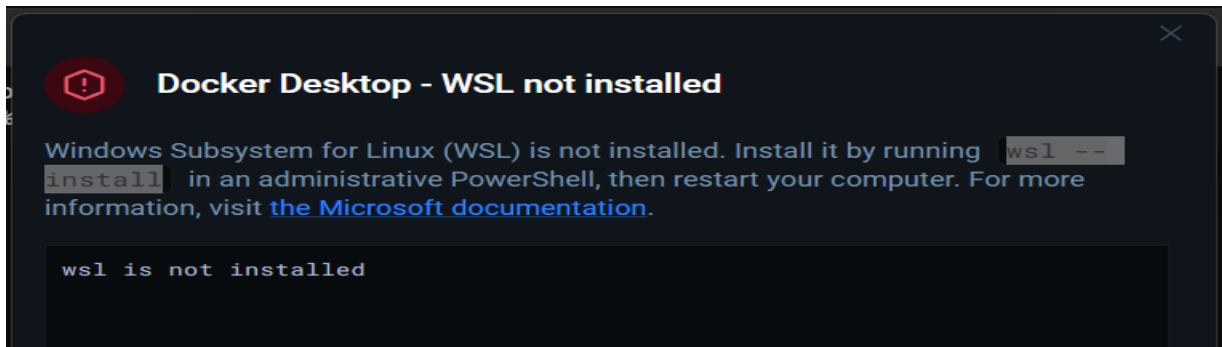
¿Abrir Docker Desktop.exe?

<https://auth.docker.com> quiere abrir esta aplicación.

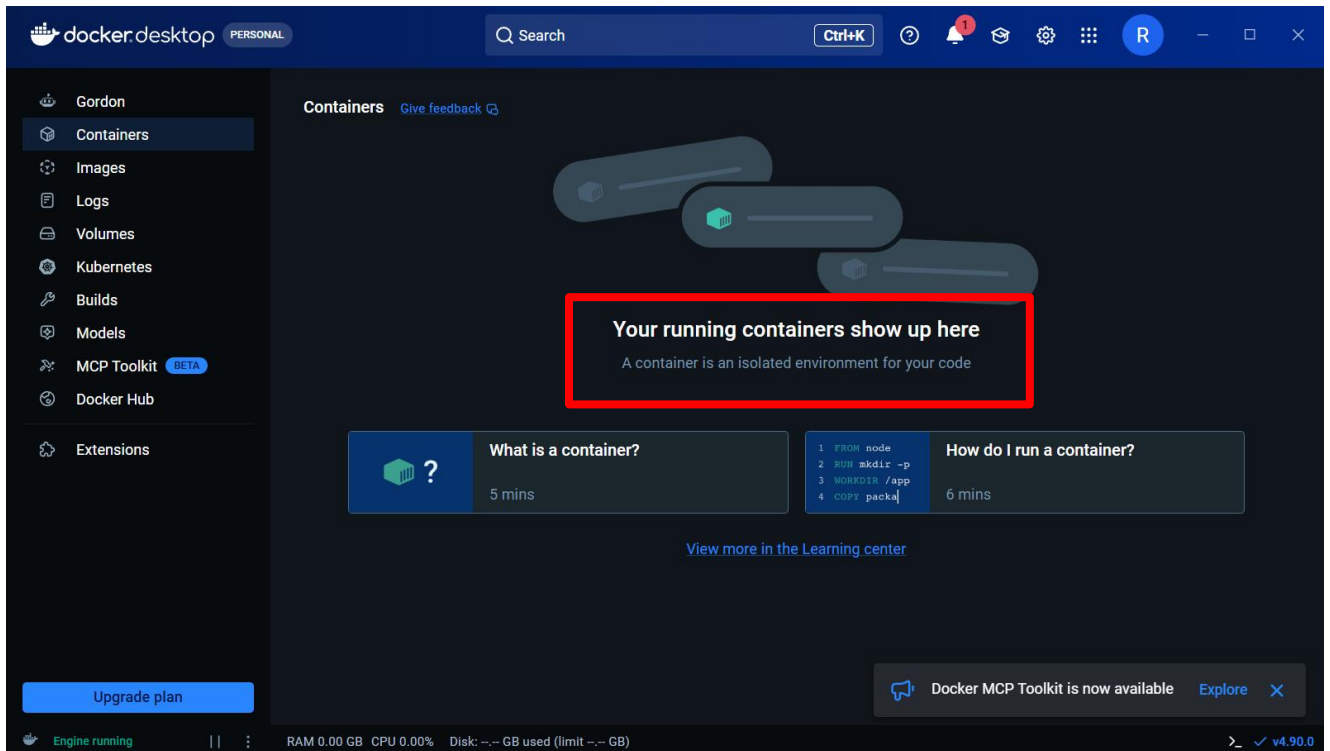
☐ Permitir siempre que auth.docker.com abra este tipo de enlaces en la aplicación asociada

Abrir Docker Desktop.exe Cancelar

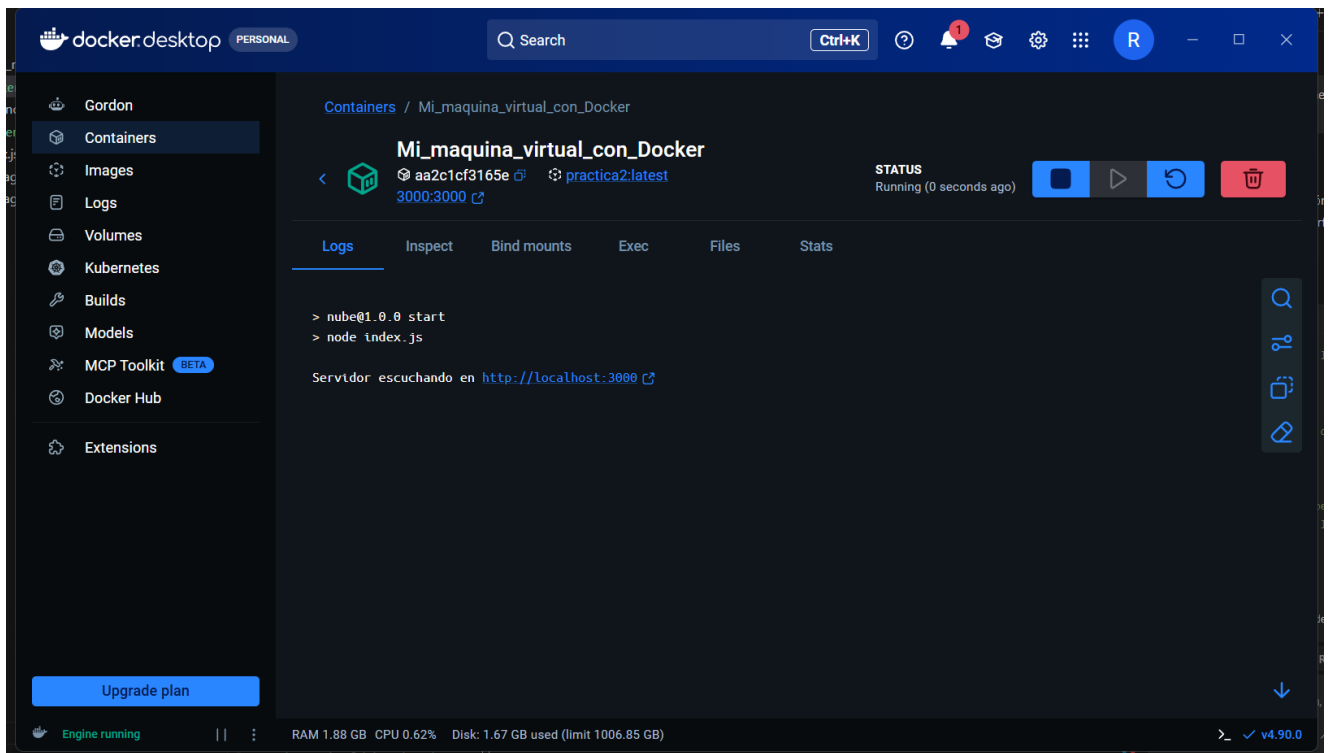
En mi caso me solicito instalar WLS, lo instalamos desde la terminal para descargar WSL, al finalizar la descarga nuevamente abrimos el programa de Docker y nos arrojarla sig ventana de WSL,



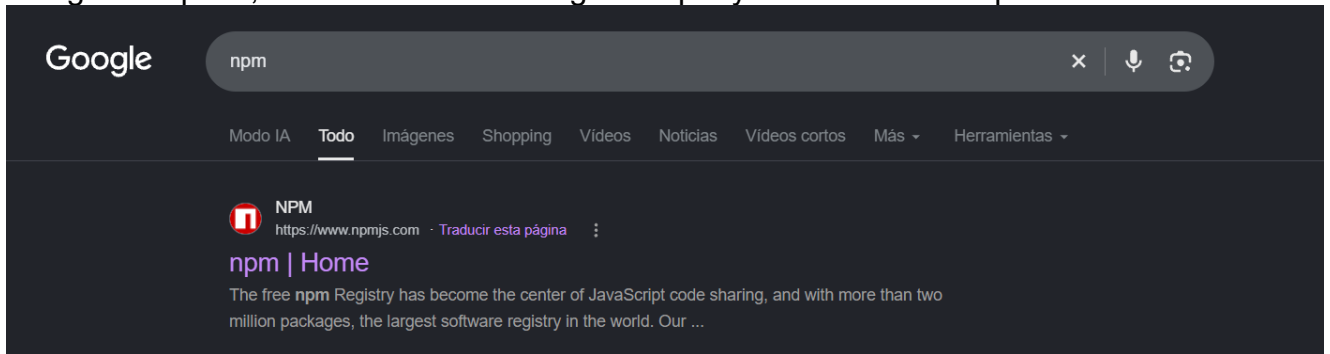
Esperamos a que inicialice el motor de Docker y este en modo “Running” y empezar a hacer nuestros primeros contenedores.



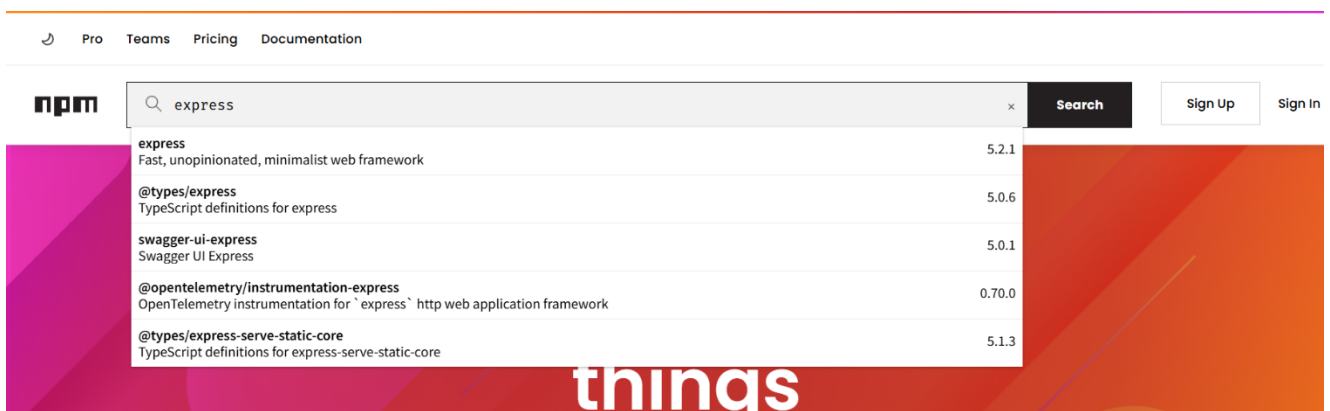
Ceamos nuestra maquina virtual en Docker



El siguiente paso, buscamos en el navegador npm y damos clic en el primer enlace



Al ingresar escribimos express en el buscador y buscamos el código para instalar desde la terminal de Node.js, este será nuestro framework de trabajo.



express DT

5.2.1 • Public • Published 10 months ago

Readme

Code Beta

28 Dependencies

108048 Dependents

289 Versions

express

Fast, unopinionated, minimalist web framework for [Node.js](#).

This project has a [Code of Conduct](#).

Table of contents

- [Table of contents](#)
- [Installation](#)

Install

> npm i express

Repository

github.com/expressjs/express

Homepage

expressjs.com/

👍 [Funding this package](#)

```
Microsoft Windows [Versión 10.0.26200.9445]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\rolda>npm i express

added 68 packages in 2s

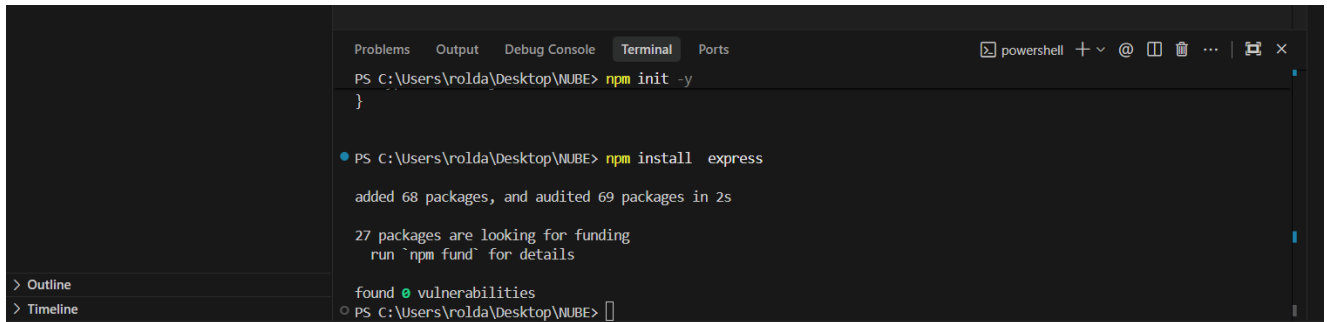
27 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New major version of npm available! 11.19.0 -> 12.0.2
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.0.2
npm notice To update run: npm install -g npm@12.0.2
npm notice

C:\Users\rolda>
```

Como siguiente paso abrimos Antigravity IDE, seleccionamos nuestra carpeta , y escribimos el siguiente codigo en la terminal **npm init -y**

```
PS C:\Users\rolda\Desktop\NUBE> npm init -y
{
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

En nuestro archivo "index.js" copiamos el siguiente código **npm install express**, esto nos creará nuestros archivos package.json.



```
PS C:\Users\rolda\Desktop\NUBE> npm init -y
}

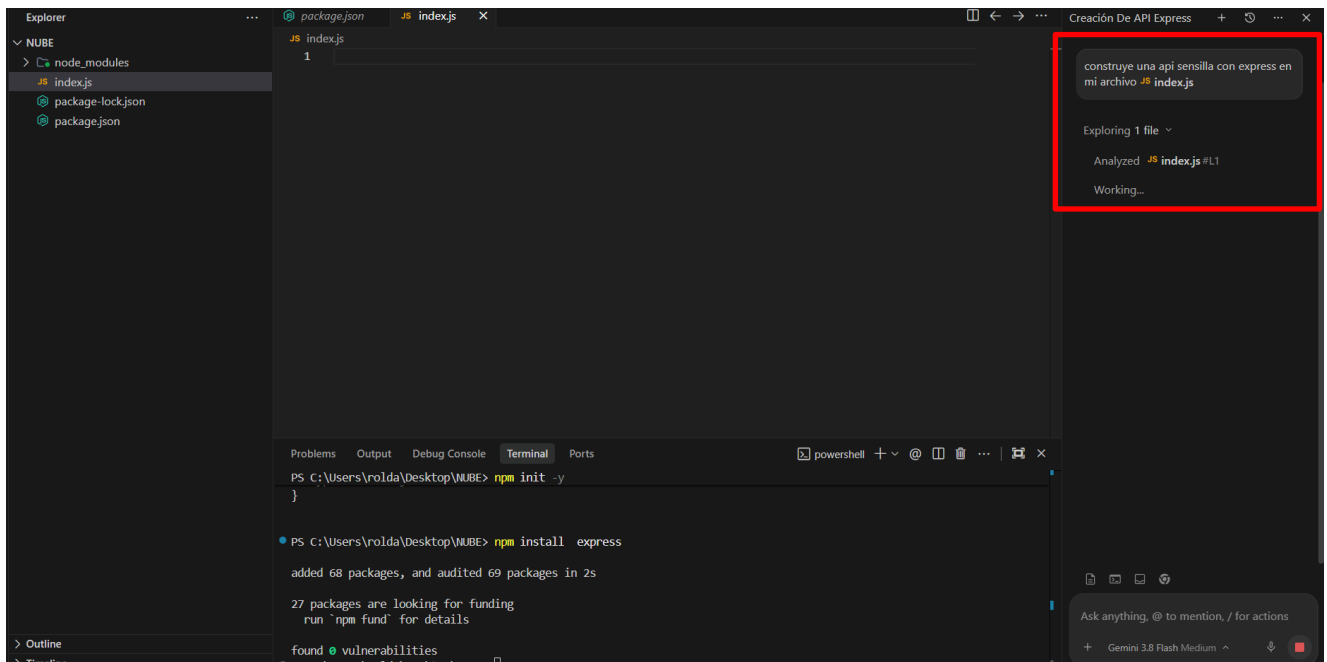
• PS C:\Users\rolda\Desktop\NUBE> npm install express

added 68 packages, and audited 69 packages in 2s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\rolda\Desktop\NUBE>
```

Creamos un archivo index.js, con ayuda del agente creamos una API con express, que corre en el puerto 3000 como se muestra en la imagen



Explorer: NUBE, node_modules, index.js, package-lock.json, package.json

index.js: 1

```
1
```

Terminal:

```
PS C:\Users\rolda\Desktop\NUBE> npm init -y
}

• PS C:\Users\rolda\Desktop\NUBE> npm install express

added 68 packages, and audited 69 packages in 2s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\rolda\Desktop\NUBE>
```

Creación De API Express

construye una api sencilla con express en mi archivo `index.js`

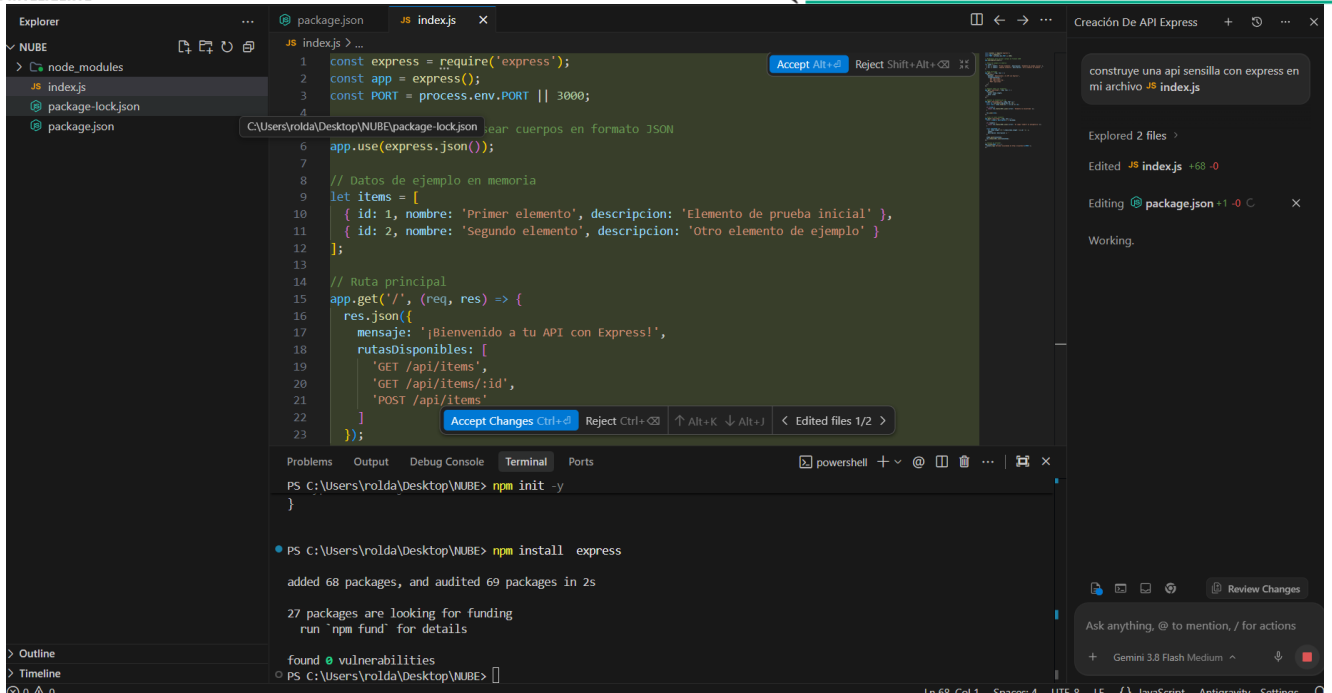
Exploring 1 file

Analyzed `index.js` #L1

Working...

Ask anything, @ to mention, / for actions

+ Gemini 3.8 Flash Medium



```

1 const express = require('express');
2 const app = express();
3 const PORT = process.env.PORT || 3000;
4
5 // Middleware para aceptar cuerpos en formato JSON
6 app.use(express.json());
7
8 // Datos de ejemplo en memoria
9 let items = [
10   { id: 1, nombre: 'Primer elemento', descripcion: 'Elemento de prueba inicial' },
11   { id: 2, nombre: 'Segundo elemento', descripcion: 'Otro elemento de ejemplo' }
12 ];
13
14 // Ruta principal
15 app.get('/', (req, res) => {
16   res.json({
17     mensaje: '¡Bienvenido a tu API con Express!',
18     rutasDisponibles: [
19       'GET /api/items',
20       'GET /api/items/:id',
21       'POST /api/items'
22     ]
23   });
24 });
  
```

```

PS C:\Users\rolida\Desktop\NUBE> npm init -y
}

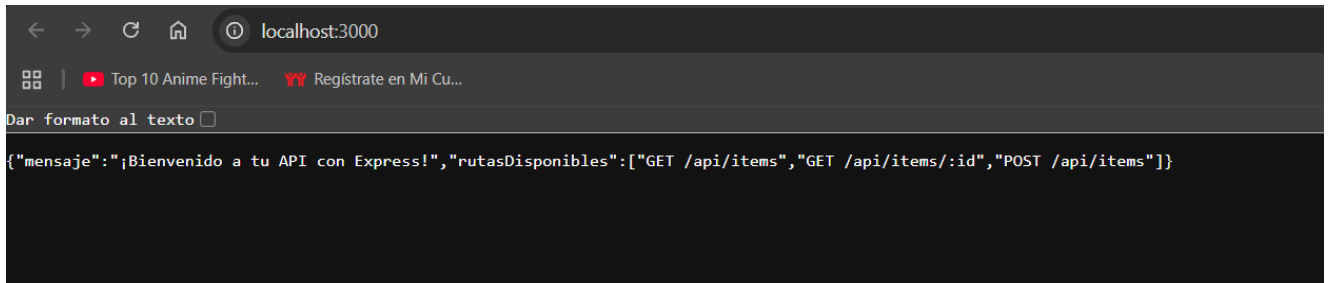
• PS C:\Users\rolida\Desktop\NUBE> npm install express

added 68 packages, and audited 69 packages in 2s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\rolida\Desktop\NUBE>
  
```

Copeamos el URL generado y comprobamos en nuestro navegador



```

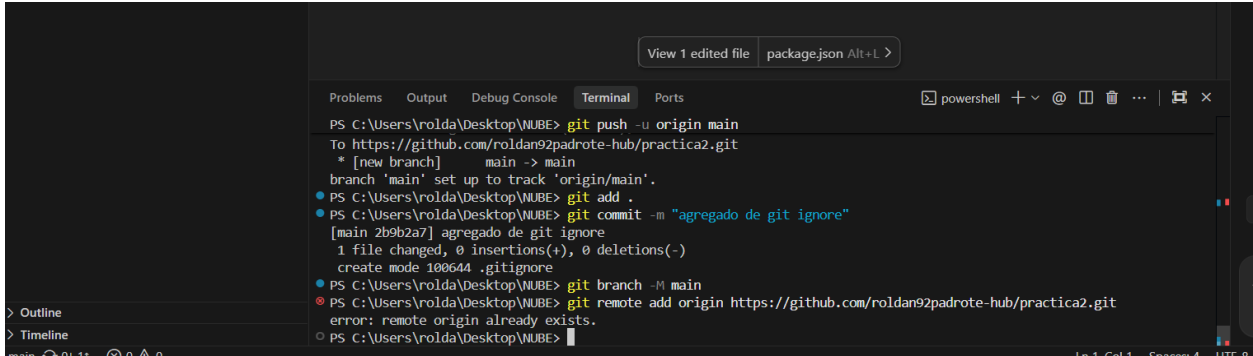
{
  "mensaje": "¡Bienvenido a tu API con Express!",
  "rutasDisponibles": [
    "GET /api/items",
    "GET /api/items/:id",
    "POST /api/items"
  ]
}
  
```

Creamos otro archivo nuevo llamado .gitignore, para no subir nuestros archivos seleccionados a Docker y colocamos los siguientes codigos en la terminal, como se muestra en la imagen, .

- 1.- ejecuta el comando git init para inicializar el repositorio local.
- 2.- Prepara todos los archivos (index.js, package.json, Dockerfile, etc.) ejecutando git add
- 3.- Realiza el primer commit ejecutando git commit -m "Despliegue de API local y Dockerfile".
- 4.- Renombra la rama principal ejecutando git branch -M main.
- 5.- Conecta tu repositorio local con el remoto ejecutando git remote add origin .

6.- Sube el código a GitHub ejecutando `git push -u origin main`.

```
echo "# practica2" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/roldan92padrote-hub/practica2.git
git push -u origin main
```



```
PS C:\Users\rolda\Desktop\NUBE> git push -u origin main
To https://github.com/roldan92padrote-hub/practica2.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\rolda\Desktop\NUBE> git add .
PS C:\Users\rolda\Desktop\NUBE> git commit -m "agregado de git ignore"
[main 2b9b2a7] agregado de git ignore
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 .gitignore
PS C:\Users\rolda\Desktop\NUBE> git branch -M main
PS C:\Users\rolda\Desktop\NUBE> git remote add origin https://github.com/roldan92padrote-hub/practica2.git
error: remote origin already exists.
PS C:\Users\rolda\Desktop\NUBE>
```

Creemos un nuevo repositorio en Github, con el nombre de la practica, privacidad publica

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

 roldan92padrote-hub /

Great repository names are short and memorable. How about **didactic-guacamole?**

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

 **Public**

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

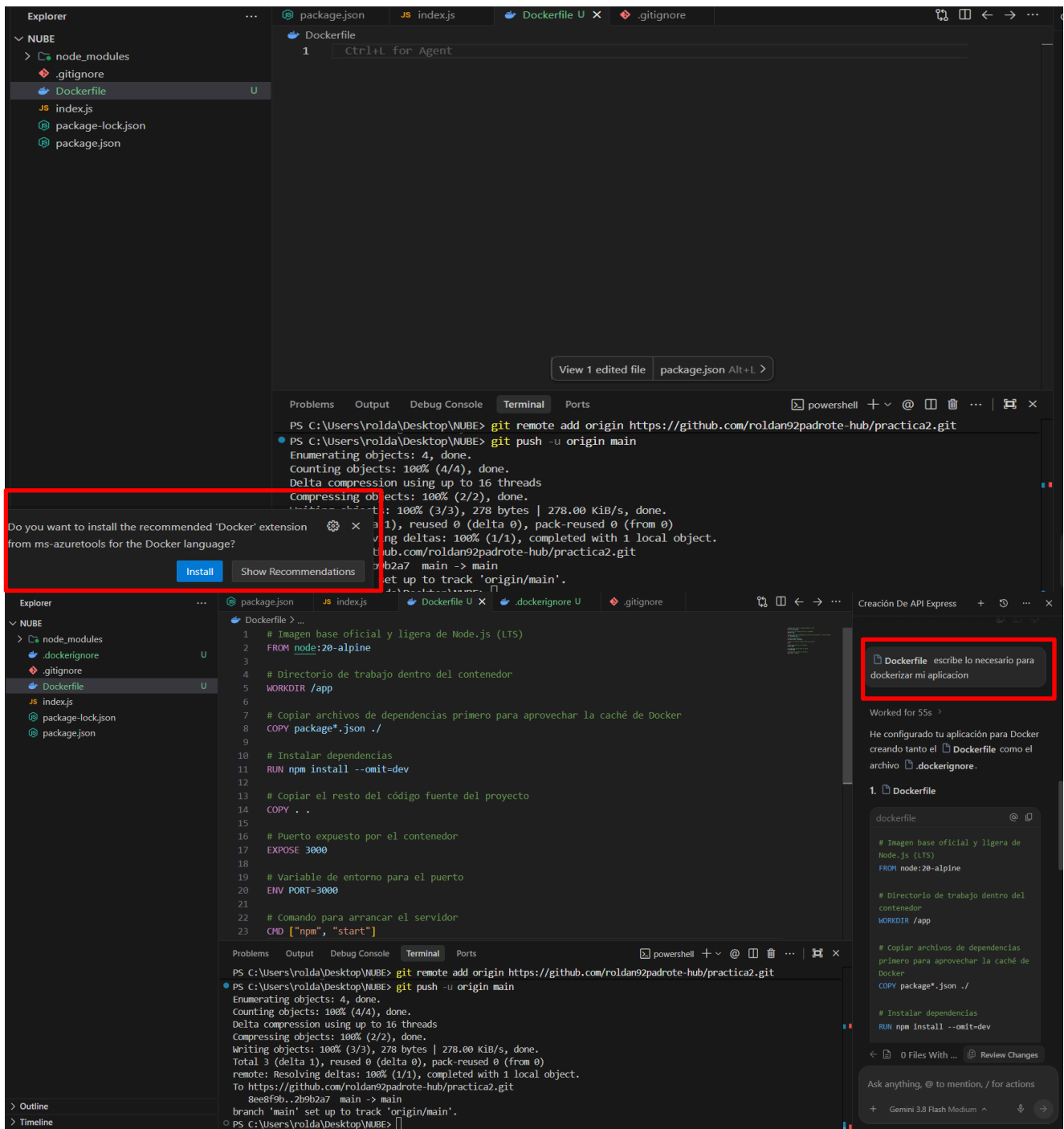
Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

El siguiente paso con ayuda del agente “dockerizamos” en la raíz de nuestra carpeta donde se encuentra nuestra API, un nuevo archivo llamado “Dockerfile” e instalamos la extension señalada.



The screenshot shows the VS Code interface with the Explorer view on the left showing the file structure of the 'NUBE' project. The Dockerfile is being created in the root of the project. The Dockerfile content is as follows:

```
1 # Imagen base oficial y ligera de Node.js (LTS)
2 FROM node:20-alpine
3
4 # Directorio de trabajo dentro del contenedor
5 WORKDIR /app
6
7 # Copiar archivos de dependencias primero para aprovechar la caché de Docker
8 COPY package*.json ./
9
10 # Instalar dependencias
11 RUN npm install --omit-dev
12
13 # Copiar el resto del código fuente del proyecto
14 COPY . .
15
16 # Puerto expuesto por el contenedor
17 EXPOSE 3000
18
19 # Variable de entorno para el puerto
20 ENV PORT=3000
21
22 # Comando para arrancar el servidor
23 CMD ["npm", "start"]
```

The terminal shows the following commands and output:

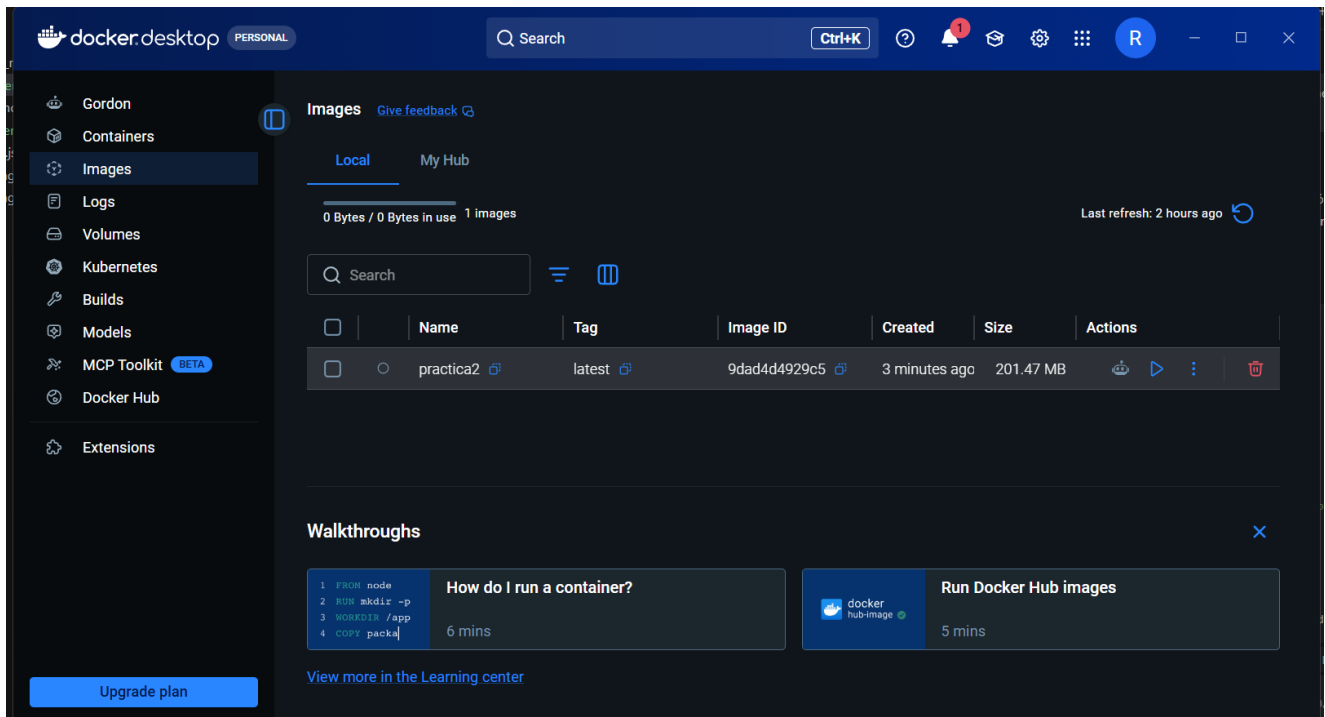
```
PS C:\Users\roldan\Desktop\NUBE> git remote add origin https://github.com/roldan92padrote-hub/practica2.git
PS C:\Users\roldan\Desktop\NUBE> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 278 bytes | 278.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/roldan92padrote-hub/practica2.git
   b2a7 main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\roldan\Desktop\NUBE>
```

A red box highlights the message: "Do you want to install the recommended 'Docker' extension from ms-azuretools for the Docker language?" with the "Install" button highlighted.

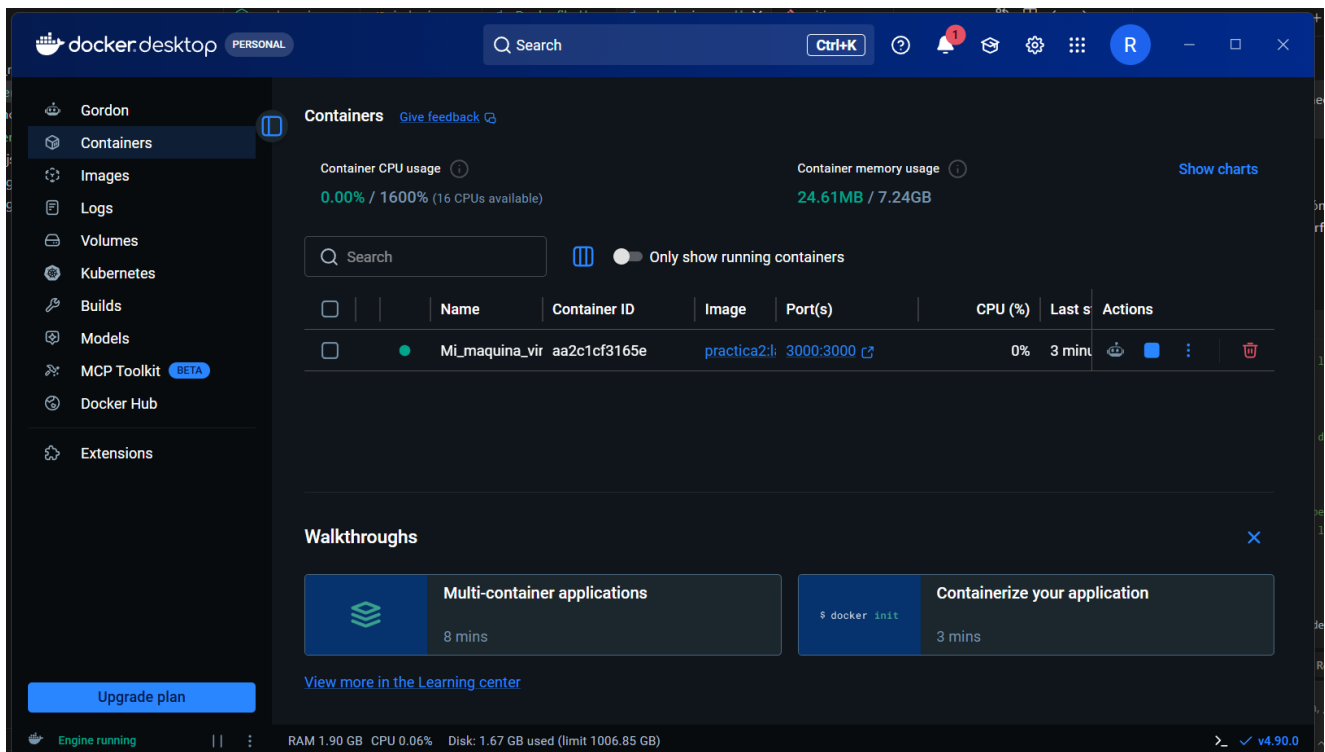
On the right, the Docker extension's welcome screen is shown, with a red box highlighting the text: "Dockerfile escribe lo necesario para dockerizar mi aplicación".

Abrimos nuestro Docker desktop, en nuestra terminal del proyecto contruimos nuestra imagen con el siguiente codigo “**docker build -t api-clase-local .**”, Una vez finalizada la construcción, ejecuta el contenedor mapeando los puertos 3000 y ejecuta **docker run -d -p**

3000:3000 --name contenedor-api api-clase-local



Presionamos el boton de RUN y validamos en nuestro navegador que este funcionando el HTTP de Docker.



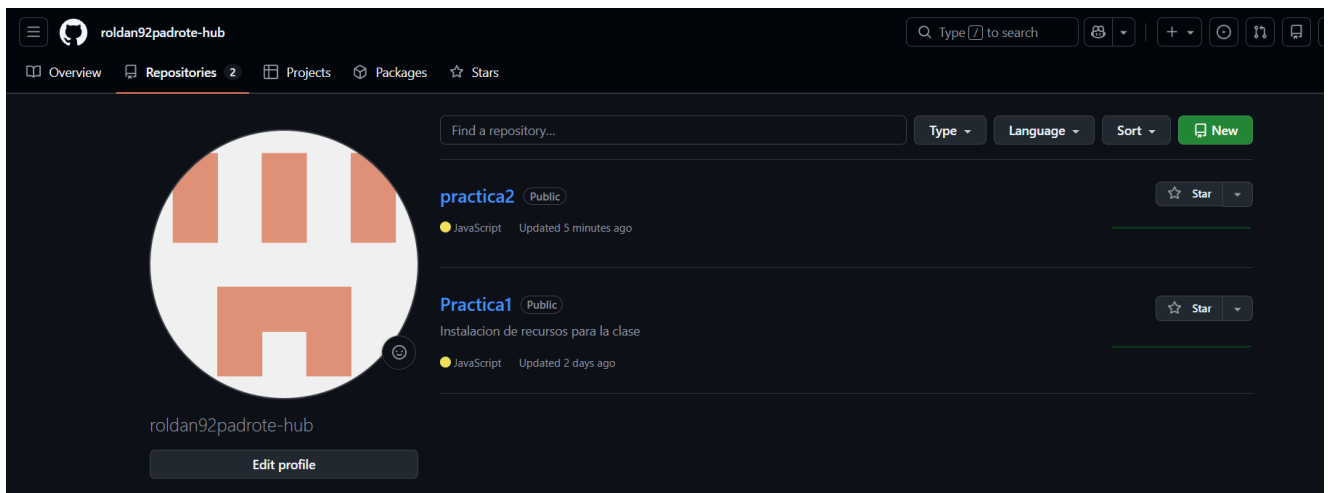
```
localhost:3000

Top 10 Anime Fight... Registré en Mi Cu...

Dar formato al texto

{"mensaje":"¡Bienvenido a tu API con Express!","rutasDisponibles":["GET /api/items","GET /api/items/:id","POST /api/items"]}
```

Iniciamos sesion en Github para crear un nuevo repositorio de nuestra prectica2



Conclusiones:

Para poder generar el contenedor de nuestra API de la practica1, se uso el programa de Docker.

Desde su intsalacion y ejecucion del codigo en Antigraivty para la correcta funcionamiento. Como utilizar el Node.js y npm express, para si poder dokerizar la API.

Ver el resultado desde nuestra maquina virtual y guardar nuestra version en Github.

Es un proceso sencillo para la comunicación con la nube de forma aisada que se deseaba.